Veritas Assignment Report

Course: LLM4ChipDesign

Instructors: Prof. Ramesh Karri

1. Overview

This report presents the implementation and evaluation of the Veritas workflow for correct-by-construction Verilog generation using a CNF-based intermediate representation. The Veritas pipeline leverages an LLM to generate Conjunctive Normal Form (CNF) specifications, which are then systematically converted into BENCH netlists, simulated to produce truth tables, and finally translated into synthesizable Verilog.

The objective of this assignment is to validate whether CNF-based generation ensures correctness through structured transformations and to evaluate the effectiveness of oracle-based validation. The workflow was applied to multiple designs, including both required tutorial runs and a scaled-up configuration.

2. Veritas Workflow

The Veritas pipeline follows a structured sequence of transformations:

1. **CNF Generation:**
   The LLM generates a CNF representation of the target logic design.
2. **CNF → BENCH Conversion:**
   The CNF clauses are translated into a BENCH netlist format.
3. **BENCH Simulation:**
   The BENCH circuit is simulated to generate a truth table (_tab.csv).
4. **Oracle Validation:**
   A Python-based oracle computes expected outputs and compares them against the generated truth table.
5. **BENCH → Verilog Conversion:**
   The verified BENCH netlist is converted into synthesizable Verilog code.

This pipeline ensures that if the CNF representation is correct, the final Verilog is guaranteed to be functionally correct.

3. Experimental Configurations

The following three configurations were executed as part of the assignment:

Run	Design	Type
Run 1	Decoder	2×4
Run 2	Adder	3-bit
Run 3 (Scale-up)	Adder	5-bit

4. Part I – Tutorial Runs

4.1 Run 1: Decoder (2×4)

Input Names:
A0, A1

Output Names:
Y0, Y1, Y2, Y3

Generated Files:

- decoder_2x4.cnf
- decoder_2x4.bench
- decoder_2x4.csv
- decoder_2x4_tab.csv
- decoder_2x4.v

Description:
The decoder takes a 2-bit input and activates exactly one of four output lines. The CNF representation encodes the one-hot output behavior.

Oracle Validation:
A Python oracle was implemented to compute the expected one-hot output based on binary input encoding.

Result:
PASS — All rows in the truth table matched the oracle output.

4.2 Run 2: Adder (3-bit)

Input Names:
A0, A1, A2, B0, B1, B2, Cin

Output Names:
S0, S1, S2, Cout

Generated Files:

- adder_3-bit.cnf
- adder_3-bit.bench
- adder_3-bit.csv
- adder_3-bit_tab.csv
- adder_3-bit.v

Description:
The design implements a 3-bit ripple-carry adder with carry-in and carry-out.

Oracle Validation:
A Python oracle computed the expected sum and carry values for all input combinations.

Result:
PASS — The generated truth table exactly matched the oracle outputs.

5. Part II – Oracle Validation

For both runs, correctness was verified using a Python-based oracle.

- The truth table (_tab.csv) generated from BENCH simulation was treated as ground output.
- The oracle computed expected outputs using binary arithmetic and logic.
- Input ordering was preserved to ensure consistent comparison.

Outcome:

- Run 1 (Decoder): PASS
- Run 2 (Adder 3-bit): PASS

No mismatches were observed, confirming that the CNF-based generation produced functionally correct designs.

6. Part III – Alternative Model Evaluation

The Veritas pipeline was executed using an alternative model (e.g., Gemini / OpenRouter-based model) instead of the default GPT-style model.

Observations:

- The alternative model required more explicit prompt structuring for CNF generation.
- Minor variations in clause formatting were observed.
- Despite these differences, the generated CNF remained logically correct.

Result:
The final outputs passed Oracle validation, demonstrating that Veritas is robust across different LLM backends.

7. Part IV – Scale-Up Experiment

Run 3: Adder (5-bit)

Input Names:
A0–A4, B0–B4, Cin

Output Names:
S0–S4, Cout

Generated Files:

- adder_5-bit.cnf
- adder_5-bit.bench
- adder_5-bit.csv
- adder_5-bit_tab.csv
- adder_5-bit.v

Observations:

- **CNF Complexity:**
  The number of clauses increased significantly compared to the 3-bit adder.
- **Runtime:**
  Generation and simulation time increased due to the larger input space.
- **Prompt Adjustments:**
  More structured prompts were used to ensure correct carry propagation.

Oracle Validation:
PASS — All truth table rows matched expected outputs.

8. Reflection: Why CNF is Effective

CNF serves as a powerful intermediate representation because it provides a precise and verifiable logical structure. By expressing the design as a conjunction of clauses, it

becomes easier to ensure logical correctness before moving to the implementation stages. Since both BENCH and Verilog are derived from the CNF, correctness propagates through the pipeline, minimizing errors introduced during later transformations. This approach shifts validation earlier in the workflow, making the overall system more reliable and scalable.

9. Conclusion

The Veritas workflow successfully generated correct Verilog designs using a CNF-based intermediate representation. All configurations passed Oracle validation, demonstrating functional correctness.

Key takeaways:

- CNF enables correctness-by-construction
- Oracle validation ensures reliability
- The pipeline scales effectively to larger designs
- The approach is robust across different LLMs

Overall, Veritas provides a structured and reliable framework for LLM-driven hardware design generation.